1 Exercise 2: General-Purpose Input-Output Ports (GPIO)

1.1 Useful References

- ATmega328p Datasheet
- AVR LIBC manual

1.2 Task 1. Port basics (Microchip Studio) (UNO)

In this task, you are going to light ON and OFF four LEDs in a sequence.

1.2.1 Building Circuit

Build the circuit using these components:

- Arduino UNO (Mega 328p)
- 4 x LEDs
- 4 x 220 Ohm resistor
- Wires

The schematic of the circuit is as follows.

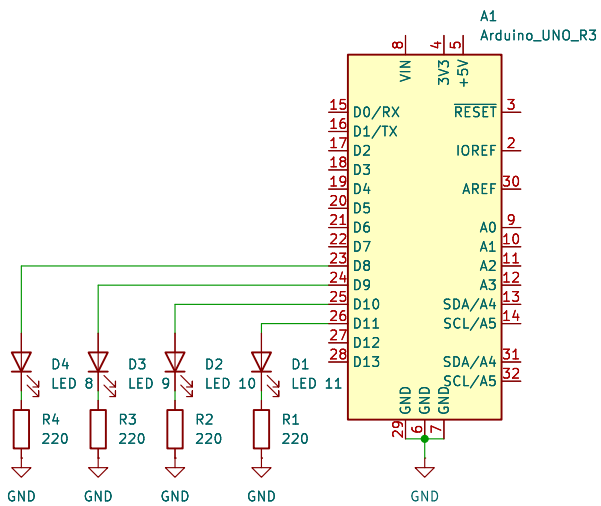

Figure 1: Circuit schematic for task 1.

- Connect positive leg of each LED with the Arduino UNO pins of 8, 9, 10, and 11.
- Connect the negative leg of each LED via the current limiting resistor 220 to the ground.

You can connect any ground pin of UNO to ground (ground bar of the bread board in practice). That is enough. However, there is 3 of them around the board edge plus the one in the 6-pin ICSP connector. If you follow the schematic and connect all the 3 (or even 4) that would not hurt. It is normal in PCB designs to have and connect multiple ground pins at once to provide shortest possible return paths for high frequency signals, i.e. signals switching at rates above 1 MHz. This helps reduving electromagnetic compatibility issues.

1.2.2 Re-using existing code

Add to project the following files from the Solution of Exercise 1:

- mcu.h for microcontroller-related defines
- uart.h and uart.c for serial port driver implemented in Exercise 1

1.2.3 Defining board and circuit mappings

Create and add to your project a file named `board_config.h`

The following snippet defines board configuration for arduino uno board didigtal IO pins 3 and 8.

```
#ifndef BOARD_CONFIG_H
#define BOARD_CONFIG_H

/// GPIO pin mappings from MCU ports to Arduino UNO digital IO pins

#define IO_3_PORT (PORTD)
#define IO_3_DIRECTION (DDRD)
#define IO_3_INPUT (PIND)
#define IO_3_PIN (PD3)

#define IO_8_PORT (PORTB)
#define IO_8_DIRECTION (DDRB)
#define IO_8_INPUT (PINB)
#define IO_8_PIN (PB0)

// Hint: add more definitions before the last endif

#endif
```

Using the pinout diagram of Arduino UNO, find the port pin of corresponding physical output pins 3, 5, 8, 9, 10, and 11. Pins 3 and 5 are used in Task 2. Add the definitions to `board_config.h`

Create and add to your project a file named `task1_pin_config.h`

The following snippet defines mapping of LED 8 to digital IO pin 8 of Arduino UNO board.

```
#ifndef TASK1_PIN_CONFIG_H
#define TASK1_PIN_CONFIG_H

#include "board_config.h"

/// GPIO pin mappings from Arduino UNO pins to task 1 circuit

#define LED_8_PORT IO_8_PORT
#define LED_8_DIRECTION IO_8_DIRECTION
#define LED_8_PIN IO_8_PIN

// Hint: add more definitions before the last endif

#endif
```

Add similar definitions for LEDs 9, 10 and 11 on pins 9, 10 and 11 respectively in `task1_pin_config.h`

1.2.4 Operating on bits

Create and add to your project a file `bit_ops.h` Write to it the following macros that will simplify setting, clearing and testing individual bits in various registers.

```

#ifndef BITOPS_H
#define BITOPS_H

/// __typeof__ preprocessor directive lets compiler detect correct type
    casting for the mask constant

/// Set bit in provided variable in place. Bit-wise OR between the current
    value and the mask
#define SET_BIT(val, bit) (val |= (((__typeof__(val))1) << bit))

```

```
/// Clear bit in provided variable in place. Bit-wise AND between the current value and the INVERTED mask
#define CLEAR_BIT(val, bit) (val &= ~(((__typeof__(val))1) << bit))

/// Read value of a single bit. Returns 1 or 0. Bitwise AND between the current value and mask is used as test.
/// Non-zero will return 1, Zero will return 0
#define READ_BIT(val, bit) ((val & (((__typeof__(val))1) << bit)) ? 1 : 0)

#endif
```

1.2.5 Task 1 main logic.

Create and add to your project a file `main_task1.c` Include the following files we have created so far and standard libraries.

```
#include "mcu.h"
#include "uart.h"

#include <avr/io.h>
#include <util/delay.h>  /*for delay*/

#include "bit_ops.h"
#include "board_config.h"
#include "task1_pin_config.h"
```

Add the following default error handler function. The purpose of this code is to unify behavior of board in any case where an unrecoverable error might occur. For now we just freeze the board to diagnose the cause.

```
/// There is not much we can do for now. This function will be improved in future.
static void handle_error(uint8_t return_code)
{
  // Non-zero return code indicates critical fault
  if (return_code)
  {
      while(1);
  }
}
```

Start writing main function by initializing the UART driver implemented in previous exercise.

```
int main(void)
{
  // Initialize serial port for standard library
        uint8_t rc = setup_uart_io();
        handle_error(rc);
```

Add initialization of LED output pins. For example LED 8 can be set as output low as follows. Add debug messages to diagnose any potential issues with serial port log.

```

  printf("Configuring IO.\r\n");

  CLEAR_BIT(LED_8_PORT, LED_8_PIN); // Default output value LOW
  SET_BIT(LED_8_DIRECTION, LED_8_PIN); // Set pin to OUTPUT

  printf("Configuration done.\r\n");
```

Write main loop toggling each led for 1 second ON separated by 1 second OFF on all LEDs. Use `SET_BIT/CLEAR_BIT` and `_delay_ms(1000);`

1.3 Task 2. Port inputs and outputs (Microchip Studio) (UNO)

In this task, you are going to light ON the LED by pressing two buttons. If one or neither of the buttons are pressed, the LED should remain OFF. Signal from buttons is used as inputs and LEDs as outputs.

1.3.1 Building Circuit

Build the circuit using these components:

- Arduino UNO (Mega 328p)
- 1 x LED
- 2 x button switches
- 1 x NPN transistor (PN2222)
- 3 x 220 Ohm resistor
- 1 x 2 kOhm resistor
- 2 x 10 kOhm resistor
- 1 x 100 kOhm resistor
- Wires

The schematic of the circuit is as follows.

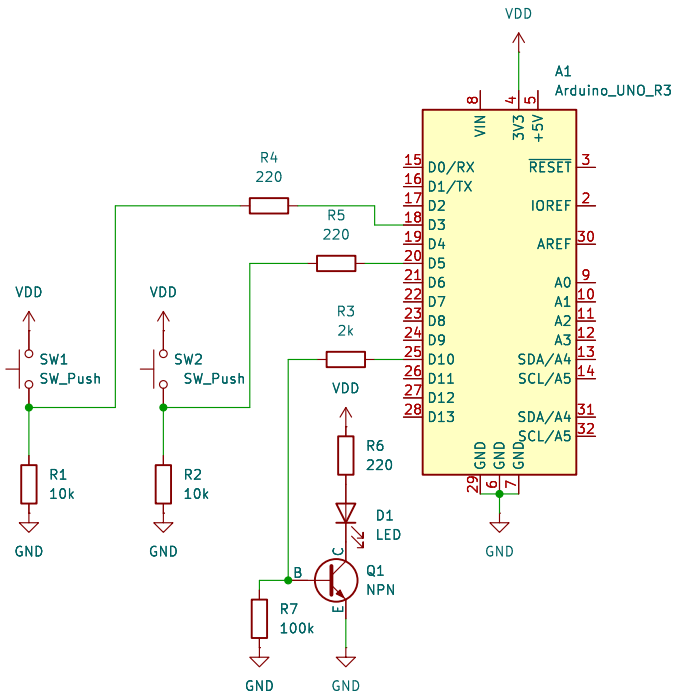

Figure 2: Circuit schematic for task 2.

In this schematic R1 and R2 are pull-down resistors which set inputs to low when buttons are not pressed. R3 and R4 are there to protect input pins from short circuit in case they are accidentally programmed as outputting low instead of input. R6 is the current limiting resistor for LED. R3 is controlling base current of transistor sized to keep it in saturation region for typical LED currents. R7 is a light pull-down on transistor base to prevent it from floating (and causing random blinking of LED) when MCU is not yet initialized.

More information on how to operate bipolar junction transistors as switches is available at Electronic Tutorials - Transistor 4

- First input button: Connect one leg to ground via a 10 kOhm resistor, and the same leg also to pin 5 of Arduino UNO via 220 Ohm resistor, while the other leg is connected to 3.3V power.
- Second input button: Connect one leg to ground via a 10 kOhm resistor, and the same leg also to pin 3 of Arduino UNO via 220 Ohm resistor, while the other leg is connected to 3.3V power.
- The LED and a current limiting 220 Ohm resistor should be in series and located between the transistor collector and the 3.3V power.
- Connect the transistor's base with 2 kOhm resistor to output in Arduino digital pin 10.
- Additionally connect the transistor's base with 100 kOhm resistor to ground.
- Connect the transistor's emitter to the ground.

1.3.2 Firmware

Here we reuse multiple files from previous work but we still need a new define for new circuit and new main function. Create a NEW project and add the following files from Task 1.

- mcu.h for microcontroller-related defines
- uart.h and uart.c for serial port driver implemented in Exercise 1
- `board_config.h` for Arduino UNO to MCU pin mapping
- `bit_ops.h` for bit operations

Create and add to your project a file named `task2_pin_config.h`
The following snippet defines mapping of BUTTON 1 to digital IO pin 3 of Arduino UNO board.

```
#ifndef TASK2_PIN_CONFIG_H
#define TASK2_PIN_CONFIG_H

#include "board_config.h"

/// GPIO pin mappings from Arduino UNO pins to task 2 circuit

#define BUTTON_1_PORT IO_3_PORT
#define BUTTON_1_DIRECTION IO_3_DIRECTION
#define BUTTON_1_INPUT IO_3_INPUT
#define BUTTON_1_PIN IO_3_PIN

// Hint: add more definitions before the last endif

#endif
```

Add similar definitions for `BUTTON_2` (pin 5) and LED (pin 10).
Create and add to project `main_task2.c` Add includes and default error handler similar to Task 1.
1) Start drafting `void main(void)` with serial and IO initialization. Clear button 1 and 2 direction bits for them to work as inputs.
2) Clear the initial value and set direction bit on LED pin to make it output low. 3) Initialize the variables for the buttons:

```
uint8_t switch_button_1 = 0; /*unsigned integer 8 bit wide*/
uint8_t switch_button_2 = 0;
```

In the while loop: 1) Read the signal from the button.

```
    printf("Loop reading inputs.\r\n");
    switch_button_1 = READ_BIT(BUTTON_1_INPUT, BUTTON_1_PIN);
    switch_button_2 = READ_BIT(BUTTON_2_INPUT, BUTTON_2_PIN);

    printf("Loop SWITCH 1: %d SWITCH 2: %d.\r\n", switch_button_1,
       switch_button_2);
```

2) Using if-else conditions, create a command to define the functionality of the buttons.

```
    if (switch_button_1 && switch_button_2)
    {
      printf("LED is ON.\r\n");
      SET_BIT(LED_PORT, LED_PIN);
    }
    else
    {
      printf("LED is OFF.\r\n");
      CLEAR_BIT(LED_PORT, LED_PIN);
    }
```

3) Put 1 second delay, using

```
_delay_ms(1000);
```

2 Using PuTTY

1. Build and upload your firmware in Microchip Studio.
2. Open PuTTY.
3. Select connection type: **Serial**.
4. Set the correct **COM port**.
5. Set **Speed** = **9600**.
6. Click **Open**.